

ArogyaSetu Bridge — Project Codebase Structure

SIH 2026 | Problem Statement 26133

1. Architecture Overview

The codebase follows a **monorepo** structure using npm workspaces, organizing the entire platform into clearly separated packages. Each package is independently buildable and deployable, while sharing common types, utilities, and configuration.

Why Monorepo? - Single source of truth for shared types (patient records, FHIR resources, API contracts) - Coordinated versioning across frontend, backend, and mobile - Simplified CI/CD pipeline management - Shared linting, formatting, and testing configuration

2. Top-Level Directory Structure

```
arogyasetu-bridge/
├── .github/                                # GitHub Actions CI/CD workflows
│   ├── workflows/
│   │   ├── ci.yml                        # Lint, test, build on every PR
│   │   ├── deploy-staging.yml           # Deploy to staging on merge to develop
│   │   ├── deploy-production.yml       # Deploy to production on release tag
│   │   └── security-scan.yml           # Dependency and container security scan
│   └── CODEOWNERS                       # Code ownership rules
├── apps/                                # Deployable applications
│   ├── web-dashboard/                  # Next.js web dashboard (providers + admin)
│   ├── mobile-app/                    # React Native mobile app (field workers + patients)
│   ├── api-gateway/                   # Express.js REST API gateway
│   ├── ml-service/                    # FastAPI ML/clinical decision support service
│   └── fhir-server/                   # HAPI FHIR server configuration
├── packages/                           # Shared libraries and utilities
│   ├── shared-types/                  # TypeScript type definitions shared across apps
│   ├── ui-components/                 # Shared React UI component library
│   ├── fhir-mapper/                   # FHIR R4 resource mapping utilities
│   ├── triage-engine/                 # Digital triage algorithm engine
│   ├── sync-engine/                   # Offline-first sync logic
│   └── notification-service/           # Multi-channel notification dispatch
├── database/                           # Database migrations and seeds
│   ├── migrations/                    # Sequenced SQL migration files
│   ├── seeds/                         # Seed data (medicines, facilities, test data)
│   └── scripts/                       # Database maintenance scripts
├── infrastructure/                     # Deployment and infrastructure config
│   ├── docker/                        # Dockerfiles for each service
│   ├── k8s/                           # Kubernetes manifests (K3s for edge)
│   ├── nginx/                         # Reverse proxy configuration
│   └── monitoring/                    # Prometheus + Grafana configs
├── docs/                               # Documentation
│   ├── api/                           # OpenAPI/Swagger specifications
│   ├── architecture/                  # Architecture decision records (ADRs)
│   ├── deployment/                    # Deployment guides
│   └── user-guides/                   # End-user documentation
├── scripts/                            # Development and utility scripts
│   ├── setup.sh                       # One-command dev environment setup
│   ├── seed-db.sh                     # Populate database with test data
│   └── generate-fhir-types.sh          # Generate TypeScript types from FHIR profiles
├── .env.example                       # Environment variable template
├── .eslintrc.json                     # Shared ESLint configuration
├── .prettierrc                        # Shared Prettier configuration
├── docker-compose.yml                 # Local development stack
├── docker-compose.prod.yml            # Production stack
├── package.json                       # Root package.json (workspaces)
├── tsconfig.base.json                 # Base TypeScript configuration
├── turbo.json                         # Turborepo pipeline configuration
├── LICENSE                            # Apache 2.0
└── README.md                          # Project overview and setup instructions
```

3. Detailed Application Structures

3.1 Web Dashboard (`apps/web-dashboard/`)

The provider-facing web dashboard built with Next.js 14, TypeScript, and React.

```

apps/web-dashboard/
├── public/
│   ├── favicon.ico
│   ├── logo.svg
│   ├── manifest.json          # PWA manifest
│   └── sw.js                  # Service Worker (generated by Workbox)
├── src/
│   ├── app/                  # Next.js App Router
│   │   ├── layout.tsx        # Root layout with providers
│   │   ├── page.tsx          # Landing / login page
│   │   └── globals.css       # Global styles
│   │
│   │   ├── (auth)/           # Authentication routes (grouped)
│   │   │   ├── login/
│   │   │   │   └── page.tsx  # OTP-based login
│   │   │   └── verify/
│   │   │       └── page.tsx  # OTP verification
│   │   │
│   │   ├── (dashboard)/      # Authenticated dashboard routes
│   │   │   ├── layout.tsx    # Dashboard layout (sidebar + topbar)
│   │   │   │
│   │   │   ├── overview/
│   │   │   │   └── page.tsx  # Facility overview dashboard
│   │   │   │
│   │   │   ├── patients/
│   │   │   │   ├── page.tsx  # Patient list with search
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx  # Patient detail view
│   │   │   │   │   ├── vitals/
│   │   │   │   │   │   └── page.tsx # Vitals history
│   │   │   │   │   ├── records/
│   │   │   │   │   │   └── page.tsx # Full health record timeline
│   │   │   │   │   └── referrals/
│   │   │   │   │       └── page.tsx # Patient referral history
│   │   │   │   └── register/
│   │   │   │       └── page.tsx  # New patient registration
│   │   │   │
│   │   │   ├── encounters/
│   │   │   │   ├── page.tsx  # Today's encounters list
│   │   │   │   ├── [id]/
│   │   │   │   │   └── page.tsx  # Encounter detail + clinical notes
│   │   │   │   └── new/
│   │   │   │       └── page.tsx  # Start new encounter
│   │   │   │
│   │   │   ├── teleconsult/
│   │   │   │   ├── page.tsx  # Teleconsult queue
│   │   │   │   ├── session/
│   │   │   │   │   ├── [id]/
│   │   │   │   │   │   └── page.tsx # Live video consultation
│   │   │   │   └── async/
│   │   │   │       └── page.tsx  # Store-and-forward requests
│   │   │   │
│   │   │   ├── triage/
│   │   │   │   ├── page.tsx  # Triage assessment list
│   │   │   │   └── new/
│   │   │   │       └── page.tsx  # New triage form (step-by-step)
│   │   │   │
│   │   │   ├── referrals/
│   │   │   │   ├── page.tsx  # Referral tracking board
│   │   │   │   ├── [id]/
│   │   │   │   │   └── page.tsx  # Referral detail with status timeline
│   │   │   │   └── incoming/

```

- └─ page.tsx # Incoming referrals (for receiving facility)
 - └─ new/
 - └─ page.tsx # Create new referral
 - └─ prescriptions/
 - └─ page.tsx # Prescription list
 - └─ new/
 - └─ page.tsx # Write prescription (linked to encounter)
 - └─ diagnostics/
 - └─ page.tsx # Diagnostic orders and results
 - └─ orders/
 - └─ page.tsx # Pending orders
 - └─ results/
 - └─ page.tsx # Completed results
 - └─ pharmacy/
 - └─ page.tsx # Pharmacy dashboard
 - └─ stock/
 - └─ page.tsx # Medicine stock levels
 - └─ dispense/
 - └─ page.tsx # Dispense medication
 - └─ alerts/
 - └─ page.tsx # Stock alerts (low / expiring)
 - └─ queue/
 - └─ page.tsx # Live queue management board
 - └─ follow-ups/
 - └─ page.tsx # Follow-up schedule overview
 - └─ overdue/
 - └─ page.tsx # Overdue follow-ups
 - └─ analytics/
 - └─ page.tsx # Analytics overview
 - └─ facility/
 - └─ page.tsx # Facility-level KPIs
 - └─ district/
 - └─ page.tsx # District-level aggregation
 - └─ state/
 - └─ page.tsx # State-level dashboard (DHO/Admin)
 - └─ settings/
 - └─ page.tsx # User profile settings
 - └─ facility/
 - └─ page.tsx # Facility configuration
 - └─ api/ # Next.js API routes (BFF layer)
 - └─ auth/
 - └─ [...nextauth]/
 - └─ route.ts # Auth handler
 - └─ proxy/
 - └─ [...path]/
 - └─ route.ts # API proxy to backend
 - └─ components/ # React components
 - └─ layout/
 - └─ Sidebar.tsx
 - └─ Topbar.tsx
 - └─ MobileNav.tsx
 - └─ BreadcrumbNav.tsx
 - └─ patients/
 - └─ PatientCard.tsx

```
├── PatientSearch.tsx
├── PatientTimeline.tsx
├── VitalsChart.tsx
├── RegistrationForm.tsx
├── encounters/
│   ├── EncounterForm.tsx
│   ├── ClinicalNotesEditor.tsx
│   ├── DiagnosisSelector.tsx # ICD-10/SNOMED CT search
│   └── EncounterSummary.tsx
├── teleconsult/
│   ├── VideoCall.tsx # WebRTC video component
│   ├── ChatPanel.tsx
│   ├── SharedScreen.tsx
│   └── ConsultNotes.tsx
├── triage/
│   ├── TriageWizard.tsx # Step-by-step symptom assessment
│   ├── UrgencyBadge.tsx
│   └── TriageResult.tsx
├── referrals/
│   ├── ReferralCard.tsx
│   ├── ReferralTimeline.tsx
│   ├── ReferralForm.tsx
│   └── FacilityPicker.tsx # Find nearest facility by service
├── pharmacy/
│   ├── StockTable.tsx
│   ├── StockAlertCard.tsx
│   ├── DispenseForm.tsx
│   └── MedicineSearch.tsx
├── queue/
│   ├── QueueBoard.tsx
│   ├── TokenDisplay.tsx
│   └── WaitTimeEstimate.tsx
├── dashboard/
│   ├── StatCard.tsx
│   ├── KPIChart.tsx
│   ├── ReferralHeatmap.tsx
│   ├── StockOverview.tsx
│   └── FacilityMap.tsx
├── common/
│   ├── Button.tsx
│   ├── Input.tsx
│   ├── Select.tsx
│   ├── Modal.tsx
│   ├── DataTable.tsx
│   ├── StatusBadge.tsx
│   ├── LoadingSpinner.tsx
│   ├── ErrorBoundary.tsx
│   ├── LanguageSwitcher.tsx
│   └── OfflineIndicator.tsx
├── hooks/ # Custom React hooks
│   ├── useAuth.ts
│   ├── usePatient.ts
│   ├── useEncounter.ts
│   ├── useReferral.ts
│   └── useQueue.ts
```

```

├── useStock.ts
├── useOfflineSync.ts
├── useTeleconsult.ts
├── useNotifications.ts
├── lib/                                # Utility libraries
│   ├── api-client.ts                 # Axios/fetch wrapper
│   ├── auth.ts                       # Auth utilities
│   ├── fhir.ts                       # FHIR helpers
│   ├── i18n.ts                       # Internationalization setup
│   ├── websocket.ts                 # Socket.IO client
│   └── offline-db.ts                 # Dexie.js database config
├── store/                             # State management (Zustand)
│   ├── auth-store.ts
│   ├── patient-store.ts
│   ├── queue-store.ts
│   ├── notification-store.ts
│   └── sync-store.ts
├── i18n/                              # Translation files
│   ├── en.json                      # English translations
│   ├── hi.json                      # Hindi translations
│   └── mr.json                      # Marathi translations
├── styles/                            # CSS modules
│   ├── variables.css                 # Design tokens
│   ├── dashboard.module.css
│   ├── forms.module.css
│   └── teleconsult.module.css
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
├── package.json
└── README.md

```

3.2 Mobile App (`apps/mobile-app/`)

React Native (Expo) application for field workers (ASHA, ANM, CHO) and patients.

```

apps/mobile-app/
├── app/                                # Expo Router (file-based routing)
│   ├── _layout.tsx                    # Root layout
│   ├── index.tsx                      # Splash / login screen
│   ├── (auth)/
│   │   ├── login.tsx                 # Phone + OTP login
│   │   └── verify.tsx                # OTP verification
│   ├── (worker)/                     # Health worker screens
│   │   ├── _layout.tsx               # Bottom tab navigator
│   │   ├── home.tsx                 # Worker dashboard / today's tasks
│   │   ├── patients/
│   │   │   ├── index.tsx            # Patient list (recently visited)
│   │   │   ├── search.tsx          # Patient search (name, ABHA, phone)
│   │   │   ├── register.tsx        # Quick patient registration
│   │   │   └── [id]/
│   │   │       ├── index.tsx        # Patient summary card
│   │   │       ├── vitals.tsx       # Record vitals (form)
│   │   │       └── history.tsx      # Patient visit history
│   │   ├── triage/
│   │   │   ├── index.tsx            # Start triage assessment
│   │   │   └── result.tsx           # Triage result + recommended action
│   │   ├── home-visits/
│   │   │   ├── index.tsx            # Today's scheduled home visits
│   │   │   ├── active.tsx           # Active home visit (data entry)
│   │   │   └── complete.tsx         # Complete and sync visit
│   │   ├── follow-ups/
│   │   │   ├── index.tsx            # Pending follow-ups list
│   │   │   └── overdue.tsx          # Overdue follow-ups (highlighted)
│   │   ├── referrals/
│   │   │   ├── index.tsx            # My referrals (sent)
│   │   │   └── track.tsx            # Track referral status
│   │   ├── teleconsult/
│   │   │   ├── request.tsx          # Request teleconsultation
│   │   │   └── session.tsx          # Assisted teleconsult (video)
│   │   └── stock/
│   │       └── check.tsx             # Check medicine availability nearby
│   ├── (patient)/                    # Patient-facing screens
│   │   ├── _layout.tsx
│   │   ├── home.tsx                 # Patient home (appointments, reminders)
│   │   ├── appointments/
│   │   │   ├── index.tsx            # My appointments
│   │   │   └── book.tsx             # Book new appointment
│   │   ├── records.tsx              # My health records
│   │   ├── medicines.tsx            # My medications + reminders
│   │   ├── facilities.tsx           # Find nearby facilities
│   │   └── emergency.tsx            # Emergency SOS screen
│   ├── (admin)/                      # Facility admin screens (CHO/Doctor)
│   │   ├── queue.tsx                # Queue management
│   │   ├── dashboard.tsx            # Facility quick stats
│   │   └── stock-update.tsx         # Quick stock update
└── components/                       # React Native components

```



```

├── common/
│   ├── Button.tsx
│   ├── TextInput.tsx
│   ├── Card.tsx
│   ├── Badge.tsx
│   ├── OfflineBanner.tsx      # Shows offline/online status
│   └── VoiceGuide.tsx        # Voice instruction component
├── vitals/
│   ├── VitalsForm.tsx        # Quick vitals entry
│   ├── BPInput.tsx           # Blood pressure specific
│   └── VitalsHistory.tsx
├── triage/
│   ├── SymptomChecklist.tsx  # Icon-driven symptom selection
│   └── UrgencyDisplay.tsx    # Color-coded urgency result
├── teleconsult/
│   ├── VideoView.tsx
│   └── PatientInfoOverlay.tsx
├── services/                  # API and local services
│   ├── api.ts                # API client (with offline queue)
│   ├── offline-db.ts         # Dexie.js local database
│   ├── sync-manager.ts       # Background sync logic
│   ├── notification-handler.ts # Push notification handler
│   └── location.ts           # GPS location services
├── utils/
│   ├── i18n.ts               # Internationalization
│   ├── permissions.ts        # Camera, location, notification permissions
│   └── connectivity.ts       # Network status monitoring
├── assets/
│   ├── icons/                # Custom icon set (for low-literacy UI)
│   ├── images/
│   └── sounds/               # Notification sounds
├── app.json                  # Expo config
├── eas.json                  # EAS Build config
├── tsconfig.json
├── package.json
└── README.md

```

3.3 API Gateway (`apps/api-gateway/`)

Express.js REST API — the central backend for all client applications.

```

apps/api-gateway/
├── src/
│   ├── index.ts           # Server entry point
│   ├── app.ts             # Express app setup (middleware, routes)
│   ├── config/
│   │   ├── database.ts    # PostgreSQL connection (Prisma)
│   │   ├── redis.ts       # Redis connection
│   │   ├── rabbitmq.ts    # RabbitMQ connection
│   │   ├── storage.ts     # MinIO/S3 config
│   │   └── environment.ts # Environment variable validation
│   ├── middleware/
│   │   ├── auth.ts        # JWT verification + RBAC
│   │   ├── rate-limiter.ts # Rate limiting (per user/IP)
│   │   ├── request-logger.ts # Request/response logging
│   │   ├── error-handler.ts # Global error handler
│   │   ├── validator.ts   # Request validation (Zod)
│   │   ├── facility-scope.ts # Facility-scoped data access
│   │   └── audit.ts       # Audit trail middleware
│   ├── routes/
│   │   ├── index.ts       # Route aggregator
│   │   ├── auth.routes.ts # Login, OTP, token refresh
│   │   ├── patient.routes.ts # CRUD + search patients
│   │   ├── encounter.routes.ts # Create/update encounters
│   │   ├── vitals.routes.ts # Record/retrieve vitals
│   │   ├── triage.routes.ts # 4-Tier CDSS Triage assessments
│   │   ├── prescription.routes.ts # e-Prescription (NMC Format Appendix 2)
│   │   ├── referral.routes.ts # Referral lifecycle & MJPJAY routing
│   │   ├── diagnostic.routes.ts # Diagnostic orders and results
│   │   ├── appointment.routes.ts # Booking and queue management
│   │   ├── pharmacy.routes.ts # FEFO stock & drug batch management
│   │   ├── teleconsult.routes.ts # Teleconsult session & consent logs
│   │   ├── asha-incentive.routes.ts # NHM incentive auto-claims & verification
│   │   ├── follow-up.routes.ts # High-risk follow-up schedules
│   │   ├── notification.routes.ts # Multi-channel notification dispatch
│   │   ├── facility.routes.ts # Facility registry and equipment
│   │   ├── analytics.routes.ts # 7 Core M&E KPI endpoints
│   │   ├── sync.routes.ts # Offline sync & CBOR delta endpoints
│   │   ├── fhir.routes.ts # FHIR R4 resource endpoints
│   │   ├── eaushadhi.routes.ts # State DVDMS inventory sync
│   │   └── abdm.routes.ts # ABDM integration (M1, M2, M3)
│   ├── controllers/
│   │   ├── auth.controller.ts
│   │   ├── patient.controller.ts
│   │   ├── encounter.controller.ts
│   │   ├── vitals.controller.ts
│   │   ├── triage.controller.ts
│   │   ├── prescription.controller.ts
│   │   ├── referral.controller.ts
│   │   ├── diagnostic.controller.ts
│   │   ├── appointment.controller.ts
│   │   ├── pharmacy.controller.ts
│   │   ├── teleconsult.controller.ts
│   │   ├── asha-incentive.controller.ts
│   │   ├── follow-up.controller.ts
│   │   ├── notification.controller.ts
│   │   ├── facility.controller.ts
│   │   ├── analytics.controller.ts
│   │   ├── sync.controller.ts
│   │   └── eaushadhi.controller.ts

```

```

└─ abdm.controller.ts

└─ services/                                # Business logic layer
  └─ auth.service.ts
  └─ patient.service.ts
  └─ encounter.service.ts
  └─ vitals.service.ts
  └─ triage.service.ts
  └─ prescription.service.ts
  └─ referral.service.ts
  └─ diagnostic.service.ts
  └─ appointment.service.ts
  └─ queue.service.ts
  └─ pharmacy.service.ts
  └─ teleconsult.service.ts
  └─ follow-up.service.ts
  └─ notification.service.ts
  └─ facility.service.ts
  └─ analytics.service.ts
  └─ sync.service.ts
  └─ fhir.service.ts
  └─ abdm.service.ts

└─ models/                                  # Prisma schema and generated client
  └─ prisma/
    └─ schema.prisma                        # Prisma schema (maps to PostgreSQL)
    └─ migrations/                          # Prisma migrations

└─ jobs/                                    # Background job processors
  └─ referral-reminder.job.ts               # Send referral follow-up reminders
  └─ stock-alert.job.ts                     # Check stock levels and alert
  └─ follow-up-checker.job.ts               # Flag overdue follow-ups
  └─ daily-stats.job.ts                     # Aggregate daily facility stats
  └─ sync-processor.job.ts                  # Process offline sync queue

└─ workers/                                # RabbitMQ consumer workers
  └─ notification.worker.ts                 # Process notification queue
  └─ fhir-export.worker.ts                  # Export records as FHIR bundles
  └─ analytics.worker.ts                    # Process analytics events

└─ websocket/                              # WebSocket handlers (Socket.IO)
  └─ index.ts                              # Socket.IO server setup
  └─ queue.handler.ts                       # Real-time queue updates
  └─ teleconsult.handler.ts                 # Teleconsult signaling
  └─ notification.handler.ts                # Real-time notifications

└─ utils/
  └─ logger.ts                             # Winston logger
  └─ encryption.ts                         # AES-256 encrypt/decrypt
  └─ otp.ts                                # OTP generation and verification
  └─ sms.ts                                # SMS sending (Twilio)
  └─ whatsapp.ts                           # WhatsApp Business API
  └─ referral-code.ts                       # Generate referral codes
  └─ validators/                            # Zod validation schemas
    └─ patient.schema.ts
    └─ encounter.schema.ts
    └─ referral.schema.ts
    └─ vitals.schema.ts
    └─ prescription.schema.ts

└─ types/
  └─ index.ts                              # Local type augmentations

└─ tests/

```

```

├── unit/
│   ├── services/
│   │   ├── patient.service.test.ts
│   │   ├── referral.service.test.ts
│   │   ├── triage.service.test.ts
│   │   └── pharmacy.service.test.ts
│   └── utils/
│       └── encryption.test.ts
├── integration/
│   ├── auth.test.ts
│   ├── patient-flow.test.ts
│   ├── referral-flow.test.ts
│   └── sync-flow.test.ts
├── fixtures/
│   ├── patients.json
│   ├── facilities.json
│   └── medicines.json
├── tsconfig.json
├── package.json
├── jest.config.ts
├── Dockerfile
└── README.md

```

3.4 ML Service (apps/ml-service/)

FastAPI-based Python service for clinical decision support and analytics.

```

apps/ml-service/
├── app/
│   ├── main.py                # FastAPI application entry
│   ├── config.py              # Environment config
│   └── routers/
│       ├── triage.py           # Triage prediction endpoints
│       ├── risk_prediction.py  # High-risk patient identification
│       ├── demand_forecast.py # Medicine demand forecasting
│       ├── clinical_decision.py # Clinical decision support
│       └── health.py           # Health check endpoint
│
│   ├── models/                # ML model definitions
│       ├── triage_classifier.py # Triage urgency classifier
│       ├── risk_scorer.py       # Patient risk scoring model
│       ├── demand_predictor.py  # Time-series medicine demand
│       └── symptom_analyzer.py  # Symptom-to-condition mapping
│
│   ├── services/
│       ├── prediction_service.py # Model inference logic
│       ├── data_service.py       # Database queries for ML
│       └── feature_engineering.py # Feature extraction from clinical data
│
│   ├── data/
│       ├── symptom_ontology.json # Structured symptom database
│       ├── icd10_codes.json      # ICD-10 code mappings
│       ├── triage_rules.json     # WHO-adapted triage decision rules
│       └── drug_interactions.json # Known drug interaction database
│
│   ├── trained_models/         # Serialized trained models
│       ├── triage_v1.pkl
│       ├── risk_scorer_v1.pkl
│       └── demand_model_v1.pkl
│
│   └── utils/
│       ├── logger.py
│       └── validators.py
│
├── tests/
│   ├── test_triage.py
│   ├── test_risk_prediction.py
│   └── test_demand_forecast.py
│
├── notebooks/                  # Jupyter notebooks for model development
│   ├── 01_triage_model_training.ipynb
│   ├── 02_risk_scoring_analysis.ipynb
│   └── 03_demand_forecasting.ipynb
│
├── requirements.txt
├── Dockerfile
├── pyproject.toml
└── README.md

```

3.5 FHIR Server (`apps/fhir-server/`)

HAPI FHIR server configuration for ABDM-compliant health record exchange.

```

apps/fhir-server/
├── config/
│   ├── application.yaml           # HAPI FHIR server configuration
│   ├── fhir-profiles/           # Custom FHIR profiles for India/ABDM
│   │   ├── patient-in.json      # Indian Patient profile
│   │   ├── encounter-in.json    # Indian Encounter profile
│   │   ├── observation-vitals.json # Vitals observation profile
│   │   ├── medication-request.json # Prescription profile
│   │   └── service-request.json  # Referral/diagnostic request profile
│   └── terminology/
│       ├── snomed-ct-subset.json # SNOMED CT subset for primary care
│       ├── icd10-india.json      # ICD-10 codes used in India
│       └── loinc-common.json     # Common LOINC codes for labs
├── Dockerfile
├── docker-compose.fhir.yml
└── README.md

```

4. Shared Packages

4.1 Shared Types (`packages/shared-types/`)

```

packages/shared-types/
├── src/
│   ├── index.ts                 # Barrel export
│   ├── patient.types.ts        # Patient interfaces
│   ├── encounter.types.ts      # Encounter interfaces
│   ├── vitals.types.ts         # Vitals interfaces
│   ├── triage.types.ts         # Triage interfaces + urgency enum
│   ├── prescription.types.ts   # Prescription interfaces
│   ├── referral.types.ts       # Referral interfaces + status enum
│   ├── diagnostic.types.ts     # Diagnostic order/result interfaces
│   ├── appointment.types.ts    # Appointment interfaces
│   ├── pharmacy.types.ts       # Medicine + stock interfaces
│   ├── facility.types.ts       # Facility interfaces + type enum
│   ├── user.types.ts           # User interfaces + role enum
│   ├── notification.types.ts   # Notification interfaces
│   ├── fhir.types.ts           # FHIR R4 resource type mappings
│   ├── api.types.ts            # API request/response types
│   └── enums.ts                # Shared enumerations
├── tsconfig.json
└── package.json

```

4.2 Triage Engine (`packages/triage-engine/`)

```
packages/triage-engine/
├── src/
│   ├── index.ts                # Engine entry point
│   ├── triage-runner.ts        # Main triage assessment runner
│   ├── symptom-checker.ts      # Symptom-to-condition mapping
│   ├── urgency-classifier.ts   # Rule-based urgency classification
│   ├── facility-matcher.ts     # Match condition to appropriate facility
│   └── rules/
│       ├── maternal.rules.ts   # Maternal health triage rules
│       ├── pediatric.rules.ts  # Pediatric triage rules
│       ├── cardiac.rules.ts    # Cardiac emergency rules
│       ├── respiratory.rules.ts # Respiratory condition rules
│       ├── general.rules.ts    # General symptom rules
│       └── chronic.rules.ts    # Chronic disease assessment
│   └── data/
│       ├── symptom-tree.json   # Hierarchical symptom questionnaire
│       └── red-flags.json      # Emergency red flag symptoms
├── tests/
│   ├── triage-runner.test.ts
│   ├── urgency-classifier.test.ts
│   └── facility-matcher.test.ts
├── tsconfig.json
└── package.json
```

4.3 Sync Engine (`packages/sync-engine/`)

```
packages/sync-engine/
├── src/
│   ├── index.ts
│   ├── offline-store.ts        # Dexie.js schema for local IndexedDB
│   ├── sync-manager.ts         # Orchestrates sync cycles
│   ├── conflict-resolver.ts    # Handles sync conflicts
│   ├── queue-manager.ts        # Manages outbound operation queue
│   ├── change-tracker.ts       # Tracks local changes for sync
│   └── schemas/
│       ├── patient.schema.ts   # Local patient table schema
│       ├── encounter.schema.ts  # Local encounter table schema
│       ├── vitals.schema.ts     # Local vitals table schema
│       └── prescription.schema.ts # Local prescription table schema
├── tests/
│   ├── sync-manager.test.ts
│   └── conflict-resolver.test.ts
├── tsconfig.json
└── package.json
```

4.4 FHIR Mapper (`packages/fhir-mapper/`)

```
packages/fhir-mapper/
├── src/
│   ├── index.ts
│   ├── patient-mapper.ts           # Patient → FHIR Patient resource
│   ├── encounter-mapper.ts        # Encounter → FHIR Encounter resource
│   ├── observation-mapper.ts      # Vitals → FHIR Observation resource
│   ├── medication-mapper.ts       # Prescription → FHIR MedicationRequest
│   ├── service-request-mapper.ts  # Referral → FHIR ServiceRequest
│   ├── diagnostic-mapper.ts       # DiagnosticOrder → FHIR DiagnosticReport
│   ├── bundle-builder.ts          # Build FHIR Bundles for exchange
│   └── validators/
│       └── fhir-validator.ts      # Validate against ABDM profiles
├── tests/
│   └── mapper.test.ts
├── tsconfig.json
└── package.json
```

5. Database Directory

```
database/
├── migrations/
│   ├── 001_create_facilities.sql
│   ├── 002_create_users.sql
│   ├── 003_create_user_sessions.sql
│   ├── 004_create_patients.sql
│   ├── 005_create_patient_vitals.sql
│   ├── 006_create_triage_assessments.sql
│   ├── 007_create_encounters.sql
│   ├── 008_create_prescriptions.sql
│   ├── 009_create_prescription_items.sql
│   ├── 010_create_referrals.sql
│   ├── 011_create_diagnostic_orders.sql
│   ├── 012_create_diagnostic_results.sql
│   ├── 013_create_facility_equipment.sql
│   ├── 014_create_medicines.sql
│   ├── 015_create_medicine_stock.sql
│   ├── 016_create_medicine_stock_transactions.sql
│   ├── 017_create_appointments.sql
│   ├── 018_create_queue_entries.sql
│   ├── 019_create_follow_up_schedules.sql
│   ├── 020_create_notifications.sql
│   ├── 021_create_facility_daily_stats.sql
│   ├── 022_create_audit_log.sql
│   ├── 023_create_sync_queue.sql
│   └── 024_create_indexes.sql
├── seeds/
│   ├── 01_facilities.sql           # Maharashtra PHCs, CHCs, DHs
│   ├── 02_medicines.sql           # Essential medicines list
│   ├── 03_test_users.sql          # Test user accounts
│   ├── 04_test_patients.sql       # Sample patient data
│   └── 05_symptom_data.sql        # Triage symptom ontology
└── scripts/
    ├── backup.sh                  # Database backup script
    ├── restore.sh                 # Database restore script
    └── analytics-views.sql        # Materialized views for dashboards
```

6. Infrastructure

```
infrastructure/
├── docker/
│   ├── Dockerfile.api           # API gateway Dockerfile
│   ├── Dockerfile.web           # Web dashboard Dockerfile
│   ├── Dockerfile.ml            # ML service Dockerfile
│   ├── Dockerfile.fhir          # FHIR server Dockerfile
│   └── Dockerfile.nginx         # Nginx reverse proxy
├── k8s/
│   ├── namespace.yaml           # arogyasetu namespace
│   ├── configmap.yaml           # Shared configuration
│   ├── secrets.yaml             # Encrypted secrets
│   ├── api-deployment.yaml       # API gateway deployment
│   ├── web-deployment.yaml       # Web dashboard deployment
│   ├── ml-deployment.yaml        # ML service deployment
│   ├── fhir-deployment.yaml      # FHIR server deployment
│   ├── postgres-statefulset.yaml # PostgreSQL StatefulSet
│   ├── redis-deployment.yaml     # Redis deployment
│   ├── rabbitmq-deployment.yaml  # RabbitMQ deployment
│   ├── ingress.yaml             # Ingress rules
│   └── hpa.yaml                 # Horizontal Pod Autoscaler
├── nginx/
│   ├── nginx.conf               # Main nginx config
│   └── conf.d/
│       ├── api.conf             # API proxy config
│       └── web.conf             # Web dashboard config
└── monitoring/
    ├── prometheus/
    │   ├── prometheus.yml        # Prometheus scrape config
    │   └── alert-rules.yml       # Alerting rules
    └── grafana/
        ├── datasources.yml       # Data source configuration
        └── dashboards/
            ├── api-health.json    # API health dashboard
            ├── facility-uptime.json # Facility uptime monitoring
            └── sync-status.json    # Offline sync monitoring
```

7. Documentation

```
docs/
├── api/
│   ├── openapi.yaml           # Full OpenAPI 3.0 specification
│   ├── postman-collection.json # Postman collection for testing
│   └── api-reference.md       # API endpoint reference
├── architecture/
│   ├── ADR-001-monorepo.md     # Why monorepo
│   ├── ADR-002-offline-first.md # Offline-first design decisions
│   ├── ADR-003-fhir-compliance.md # FHIR R4 compliance approach
│   ├── ADR-004-sync-strategy.md # Sync conflict resolution strategy
│   ├── ADR-005-auth-model.md   # Authentication architecture
│   ├── system-overview.md      # High-level system architecture
│   └── data-flow.md            # Detailed data flow documentation
├── deployment/
│   ├── local-setup.md          # Local development setup guide
│   ├── staging-deployment.md    # Staging deployment procedure
│   ├── production-deployment.md # Production deployment guide
│   └── abdm-certification.md    # ABDM sandbox certification steps
└── user-guides/
    ├── asha-worker-guide.md     # Guide for ASHA workers
    ├── anm-guide.md             # Guide for ANMs
    ├── doctor-guide.md          # Guide for doctors
    ├── pharmacist-guide.md       # Guide for pharmacists
    ├── dho-admin-guide.md       # Guide for DHO/administrators
    └── patient-guide.md         # Guide for patients
```

8. Development Environment Setup

Local Development Stack (docker-compose.yml)

The `docker-compose.yml` at the root spins up the complete development environment:

Service	Port	Description
postgres	5432	PostgreSQL 16 database
redis	6379	Redis cache and queue
rabbitmq	5672 / 15672	RabbitMQ message broker (+ management UI)
minio	9000 / 9001	MinIO object storage (+ console)
elasticsearch	9200	Elasticsearch for full-text search
api-gateway	4000	Express.js API
web-dashboard	3000	Next.js web dashboard
ml-service	8000	FastAPI ML service
fhir-server	8080	HAPI FHIR R4 server
prometheus	9090	Metrics collection
grafana	3001	Monitoring dashboards

Quick Start Commands

```
# Clone and setup
git clone https://github.com/arogyasetu-bridge/arogyasetu-bridge.git
cd arogyasetu-bridge

# Copy environment file
cp .env.example .env

# Start all services
docker-compose up -d

# Run database migrations
npm run db:migrate

# Seed with test data
npm run db:seed

# Start development servers (all apps)
npm run dev

# Run tests
npm run test

# Run linting
npm run lint
```

9. Key File Count Summary

Component	Files (approx.)	Lines of Code (est.)
Web Dashboard (Next.js)	~120	~18,000
Mobile App (React Native)	~80	~12,000
API Gateway (Express.js)	~100	~15,000
ML Service (FastAPI)	~30	~4,000
FHIR Server Config	~15	~2,000
Shared Packages	~40	~6,000
Database Migrations & Seeds	~30	~3,000
Infrastructure (Docker/K8s)	~25	~1,500
Documentation	~20	~5,000
Tests	~50	~8,000
Total	~510	~74,500

Project codebase structure designed for ArogyaSetu Bridge — SIH 2026, PS 26133
Monorepo with npm workspaces, Turborepo for build orchestration, and Docker for containerized deployment.